Below are the **JSON structures + JavaScript DOCX generators** for templates **1, 2, 10, 8, 7** (matching your reference style: tables, headings, colored statuses, returns `{ blob, filename }`).

Template 1) Incident Resolution Summary (Customer-Friendly)

JSON

```
{
  "incident_number": "INC0012345",
  "short_description": "Users unable to login",
  "priority": "P2",
  "service": "Customer Portal",
  "assignment_group": "Service Desk",
  "opened_at": "2026-04-20 09:12",
  "resolved_at": "2026-04-20 11:05",
  "created_date": "2026-04-25",
  "author": "Your Name",
  "impact": {
    "impact_start": "2026-04-20 09:05",
    "impact_end": "2026-04-20 11:05",
    "affected_users": "All external users",
    "business_impact": "Users could not submit requests",
    "customer_visible": "Yes"
  },
  "symptoms": ["Login redirects to error page"],
  "resolution": {
    "summary": "Restarted IdP and cleared cache",
    "workaround": "Use direct app URL",
    "root_cause_if_known": "Expired signing cert cached"
  },
  "resolution_steps": [
    { "step": "Validated IdP logs", "owner": "Platform" }
  ],
  "next_steps": [
    { "action": "Add cert expiry monitoring", "owner": "IAM", "due_date": "2026-
05-05", "status": "Planned" }
  ],
  "related_records": { "problem": "PRB0010203", "change": "CHG0032109", "kb":
"KB0010099" }
}
```

JS generator

```
async function generateIncidentResolutionSummary(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell } =
```

```
makeDocHelpers();

  const inc = safeStr(data.incident_number || "INC");
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
  const author = safeStr(data.author || "Author");

  const impact = data.impact || {};
  const symptoms = data.symptoms || [];
  const res = data.resolution || {};
  const resSteps = data.resolution_steps || [];
  const nextSteps = data.next_steps || [];
  const rel = data.related_records || {};

  const children = [
    h1(`${inc} - Incident Resolution Summary`),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para(""),

    h2("Incident Details")
  ];

  children.push(new Table({
    width: { size: 9360, type: WidthType.DXA },
    columnWidths: [2500, 6860],
    rows: [
      new TableRow({ children: [hdrCell("Field", 2500), hdrCell("Value", 6860)]
}),
      new TableRow({ children: [dataCell("Short Description", 2500),
dataCell(data.short_description || "", 6860)] }),
      new TableRow({ children: [dataCell("Priority", 2500),
statusCell(data.priority || "", 6860)] }),
      new TableRow({ children: [dataCell("Service", 2500), dataCell(data.service
|| "", 6860)] }),
      new TableRow({ children: [dataCell("Assignment Group", 2500),
dataCell(data.assignment_group || "", 6860)] }),
      new TableRow({ children: [dataCell("Opened At", 2500),
dataCell(data.opened_at || "", 6860)] }),
      new TableRow({ children: [dataCell("Resolved At", 2500),
dataCell(data.resolved_at || "", 6860)] })
    ]
  }), para(""));

  children.push(h2("Impact"));
  children.push(...bullets([
    `Impact Window: ${safeStr(impact.impact_start)} →
${safeStr(impact.impact_end)}`,
    `Affected Users: ${safeStr(impact.affected_users)}`,
    `Customer Visible: ${safeStr(impact.customer_visible)}`,
    `Business Impact: ${safeStr(impact.business_impact)}`
  ]));

  if (symptoms.length) {
```

```
    children.push(para(""), h2("Symptoms Observed"), ...bullets(symptoms));
  }

  children.push(para(""), h2("Resolution"));
  children.push(...bullets([
    `Summary: ${safeStr(res.summary)}`,
    res.root_cause_if_known ? `Root Cause (if known):
${safeStr(res.root_cause_if_known)}` : "",
    res.workaround ? `Workaround: ${safeStr(res.workaround)}` : ""
  ].filter(Boolean)));

  if (resSteps.length) {
    children.push(para(""), h2("Resolution Steps"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [800, 6160, 2400],
      rows: [
        new TableRow({ children: [hdrCell("#", 800), hdrCell("Step", 6160),
hdrCell("Owner", 2400)] }),
        ...resSteps.map((s, i) => new TableRow({
          children: [
            dataCell(String(i + 1), 800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.step || "", 6160, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.owner || "", 2400, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }

  if (nextSteps.length) {
    children.push(para(""), h2("Next Steps / Actions"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [5000, 1800, 1400, 1160],
      rows: [
        new TableRow({ children: [hdrCell("Action", 5000), hdrCell("Owner", 1800),
hdrCell("Due Date", 1400), hdrCell("Status", 1160)] }),
        ...nextSteps.map((a, i) => new TableRow({
          children: [
            dataCell(a.action || "", 5000, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.owner || "", 1800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.due_date || "", 1400, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(a.status || "", 1160)
          ]
        }))
      ]
    }));
  }

  children.push(para(""), h2("Related Records"));
  children.push(...bullets([
    rel.problem ? `Problem: ${safeStr(rel.problem)}` : "",
    rel.change ? `Change: ${safeStr(rel.change)}` : "",
```

```
    rel.kb ? `Knowledge: ${safeStr(rel.kb)}` : ""
  ].filter(Boolean)));

  const doc = new Document({ sections: [{ properties: {}, children }] });
  const blob = await Packer.toBlob(doc);
  return { blob, filename: `${inc}_Incident_Resolution_Summary.docx`.replace(/[^a-z0-9._-]/gi, "_") };
}
```

Template 2) Major Incident Communications Pack

JSON

```
{
  "major_incident_number": "MI0000456",
  "incident_number": "INC0012345",
  "title": "Portal login failures",
  "severity": "P1",
  "service": "Customer Portal",
  "status": "In Progress",
  "created_date": "2026-04-25",
  "author": "Your Name",

  "impact": {
    "start": "2026-04-20 09:05",
    "current_status": "Login failures for all external users",
    "affected_users": "All external users",
    "business_impact": "Users cannot submit requests"
  },

  "bridge": {
    "bridge_link": "https://meet...",
    "bridge_id": "123-456",
    "comms_channel": "#mi-portal",
    "next_update_eta": "Every 30 minutes"
  },

  "initial_notification": {
    "subject": "[P1] Portal login issue - Investigation in progress",
    "message": "We are investigating login failures affecting external users..."
  },

  "updates": [
    {
      "update_no": 1,
      "time": "09:30",
      "status": "Investigating",
      "message": "Issue identified in IdP; mitigation in progress",
      "next_update_time": "10:00"
```

```
    }
  ],

  "resolution_notice": {
    "time": "11:05",
    "subject": "[Resolved] Portal login issue",
    "message": "Service restored. Root cause under investigation..."
  },

  "distribution_list": [
    "customer-it@customer.com",
    "servicedesk@yourcompany.com"
  ]
}
```

JS generator

```
async function generateMajorIncidentCommunicationsPack(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell } =
makeDocHelpers();

  const mi = safeStr(data.major_incident_number || "MI");
  const inc = safeStr(data.incident_number || "");
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
  const author = safeStr(data.author || "Author");

  const impact = data.impact || {};
  const bridge = data.bridge || {};
  const init = data.initial_notification || {};
  const updates = data.updates || [];
  const res = data.resolution_notice || {};
  const dist = data.distribution_list || [];

  const children = [
    h1(`${mi} - Major Incident Communications Pack`),
    para(`Incident: ${inc}`, false, C.GRAY, 18),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para("")
  ];

  children.push(h2("Major Incident Overview"));
  children.push(new Table({
    width: { size: 9360, type: WidthType.DXA },
    columnWidths: [2600, 6760],
    rows: [
      new TableRow({ children: [hdrCell("Field", 2600), hdrCell("Value", 6760)]
}),
      new TableRow({ children: [dataCell("Title", 2600), dataCell(data.title ||
```

```
"", 6760)] }),
      new TableRow({ children: [dataCell("Severity", 2600),
statusCell(data.severity || "", 6760)] }),
      new TableRow({ children: [dataCell("Service", 2600), dataCell(data.service
|| "", 6760)] }),
      new TableRow({ children: [dataCell("Status", 2600), statusCell(data.status
|| "", 6760)] }),
      new TableRow({ children: [dataCell("Impact Start", 2600),
dataCell(impact.start || "", 6760)] }),
      new TableRow({ children: [dataCell("Current Impact", 2600),
dataCell(impact.current_status || "", 6760)] }),
      new TableRow({ children: [dataCell("Affected Users", 2600),
dataCell(impact.affected_users || "", 6760)] }),
      new TableRow({ children: [dataCell("Business Impact", 2600),
dataCell(impact.business_impact || "", 6760)] })
    ]
  }));

  children.push(para(""), h2("Bridge / Comms Details"));
  children.push(...bullets([
    `Bridge Link: ${safeStr(bridge.bridge_link)}`,
    `Bridge ID: ${safeStr(bridge.bridge_id)}`,
    `Comms Channel: ${safeStr(bridge.comms_channel)}`,
    `Next Update ETA: ${safeStr(bridge.next_update_eta)}`
  ].filter(x => !x.endsWith(": "))));

  children.push(para(""), h2("Initial Notification"));
  children.push(para(`Subject: ${safeStr(init.subject)}`, true));
  children.push(para(safeStr(init.message)));

  if (updates.length) {
    children.push(para(""), h2("Update Messages"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [800, 1200, 1400, 4960, 1000],
      rows: [
        new TableRow({
          children: [
            hdrCell("#", 800),
            hdrCell("Time", 1200),
            hdrCell("Status", 1400),
            hdrCell("Message", 4960),
            hdrCell("Next", 1000)
          ]
        }),
        ...updates.map((u, i) => new TableRow({
          children: [
            dataCell(u.update_no != null ? String(u.update_no) : String(i + 1),
800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(u.time || "", 1200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(u.status || "", 1400),
            dataCell(u.message || "", 4960, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(u.next_update_time || "", 1000, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
```

```
            ]
          }))
        ]
      }));
    }

    children.push(para(""), h2("Resolution Notice"));
    children.push(para(`Time: ${safeStr(res.time)}`, false, C.GRAY, 18));
    children.push(para(`Subject: ${safeStr(res.subject)}`, true));
    children.push(para(safeStr(res.message)));

    if (dist.length) {
      children.push(para(""), h2("Distribution List"), ...bullets(dist));
    }

    const doc = new Document({ sections: [{ properties: {}, children }] });
    const blob = await Packer.toBlob(doc);
    return { blob, filename: `${mi}_MI_Communications_Pack.docx`.replace(/[^a-z0-
9._-]/gi, "_") };
  }
```

Template 10) Weekly Operations / Incident Trends Report

JSON

```
{
  "client_name": "Customer A",
  "report_period": "2026-04-15 to 2026-04-21",
  "created_date": "2026-04-25",
  "author": "Your Name",
  "executive_summary": "Overall incident volume stable; SLA breaches reduced vs
last week.",

  "kpis": [
    { "metric": "New Incidents", "current": "42", "target": "≤ 45", "status": "On
Track" },
    { "metric": "SLA Compliance", "current": "96%", "target": "95%", "status": "On
Track" },
    { "metric": "MTTR", "current": "3.2h", "target": "3.0h", "status": "Behind" }
  ],

  "by_priority": [
    { "priority": "P1", "count": 1 },
    { "priority": "P2", "count": 6 },
    { "priority": "P3", "count": 20 },
    { "priority": "P4", "count": 15 }
  ],
```

```
  "top_categories": [
    { "name": "Authentication", "count": 10 },
    { "name": "Network", "count": 8 }
  ],

  "top_services": [
    { "name": "Customer Portal", "count": 14 },
    { "name": "Email", "count": 7 }
  ],

  "major_incidents": [
    { "number": "MI0000456", "summary": "Portal login outage", "duration": "2h",
"rca_status": "In Progress" }
  ],

  "aging_backlog": [
    { "bucket": "0-2 days", "count": 6 },
    { "bucket": "3-7 days", "count": 4 },
    { "bucket": "8-14 days", "count": 2 },
    { "bucket": "15+ days", "count": 1 }
  ],

  "actions_next_week": [
    { "action": "Tune auth monitoring alerts", "owner": "IAM", "due_date": "2026-
04-30", "status": "In Progress" }
  ]
}
```

JS generator

```
async function generateWeeklyOpsIncidentTrendsReport(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell } =
makeDocHelpers();

  const client = safeStr(data.client_name || "Client");
  const period = safeStr(data.report_period || "");
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
  const author = safeStr(data.author || "Author");

  const kpis = data.kpis || [];
  const byPriority = data.by_priority || [];
  const topCat = data.top_categories || [];
  const topSvc = data.top_services || [];
  const mis = data.major_incidents || [];
  const aging = data.aging_backlog || [];
  const actions = data.actions_next_week || [];

  const children = [
```

```
    h1(`Weekly Operations Report - ${client}`),
    para(`Period: ${period}`, false, C.GRAY, 18),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para(""),
    h2("Executive Summary"),
    para(data.executive_summary || "")
  ];

  if (kpis.length) {
    children.push(para(""), h2("Key KPIs"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [3200, 1800, 1800, 2560],
      rows: [
        new TableRow({ children: [hdrCell("Metric", 3200), hdrCell("Current",
1800), hdrCell("Target", 1800), hdrCell("Status", 2560)] }),
        ...kpis.map((k, i) => new TableRow({
          children: [
            dataCell(k.metric || "", 3200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(k.current || "", 1800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(k.target || "", 1800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(k.status || "", 2560)
          ]
        }))
      ]
    }));
  }

  if (byPriority.length) {
    children.push(para(""), h2("Incidents by Priority"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [3000, 6360],
      rows: [
        new TableRow({ children: [hdrCell("Priority", 3000), hdrCell("Count",
6360)] }),
        ...byPriority.map((p, i) => new TableRow({
          children: [
            statusCell(p.priority || "", 3000),
            dataCell(p.count != null ? String(p.count) : "", 6360, i % 2 ?
C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }

  if (topCat.length) {
    children.push(para(""), h2("Top Categories"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [6500, 2860],
      rows: [
```

```
        new TableRow({ children: [hdrCell("Category", 6500), hdrCell("Count",
2860)] }),
        ...topCat.map((c, i) => new TableRow({
          children: [
            dataCell(c.name || "", 6500, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(c.count != null ? String(c.count) : "", 2860, i % 2 ?
C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }

  if (topSvc.length) {
    children.push(para(""), h2("Top Services"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [6500, 2860],
      rows: [
        new TableRow({ children: [hdrCell("Service", 6500), hdrCell("Count",
2860)] }),
        ...topSvc.map((s, i) => new TableRow({
          children: [
            dataCell(s.name || "", 6500, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(s.count != null ? String(s.count) : "", 2860, i % 2 ?
C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }

  if (mis.length) {
    children.push(para(""), h2("Major Incidents"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [1600, 5200, 1400, 1160],
      rows: [
        new TableRow({ children: [hdrCell("Number", 1600), hdrCell("Summary",
5200), hdrCell("Duration", 1400), hdrCell("RCA Status", 1160)] }),
        ...mis.map((m, i) => new TableRow({
          children: [
            dataCell(m.number || "", 1600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(m.summary || "", 5200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(m.duration || "", 1400, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(m.rca_status || "", 1160)
          ]
        }))
      ]
    }));
  }

  if (aging.length) {
    children.push(para(""), h2("Backlog Aging"));
```

```
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [6500, 2860],
      rows: [
        new TableRow({ children: [hdrCell("Bucket", 6500), hdrCell("Count", 2860)]
}),
        ...aging.map((a, i) => new TableRow({
          children: [
            dataCell(a.bucket || "", 6500, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.count != null ? String(a.count) : "", 2860, i % 2 ?
C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }

  if (actions.length) {
    children.push(para(""), h2("Actions for Next Week"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [5200, 1600, 1400, 1160],
      rows: [
        new TableRow({ children: [hdrCell("Action", 5200), hdrCell("Owner", 1600),
hdrCell("Due", 1400), hdrCell("Status", 1160)] }),
        ...actions.map((a, i) => new TableRow({
          children: [
            dataCell(a.action || "", 5200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.owner || "", 1600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(a.due_date || "", 1400, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            statusCell(a.status || "", 1160)
          ]
        }))
      ]
    }));
  }

  const doc = new Document({ sections: [{ properties: {}, children }] });
  const blob = await Packer.toBlob(doc);
  return { blob, filename: `${client}_Weekly_Ops_Report.docx`.replace(/[^a-z0-
9._-]/gi, "_") };
}
```

Template 8) Integration Specification (Inbound/Outbound)

JSON

```
{
  "integration_name": "ServiceNow -> Jira Incident Sync",
  "direction": "Outbound",
  "version": "1.0",
  "created_date": "2026-04-25",
  "author": "Your Name",

  "systems": { "source": "ServiceNow", "target": "Jira" },
  "environments": ["DEV", "TEST", "PROD"],

  "authentication": {
    "method": "OAuth 2.0",
    "details": "Client credentials flow; token stored in SN credential record"
  },

  "endpoints": [
    { "name": "Create Issue", "method": "POST", "url": "https://jira/api/issues",
"purpose": "Create issue from INC" }
  ],

  "field_mapping": [
    { "source_field": "incident.short_description", "target_field":
"jira.summary", "transform": "truncate(255)", "notes": "Required" }
  ],

  "error_handling": [
    { "scenario": "401 Unauthorized", "behavior": "Refresh token and retry once",
"owner_action": "Check credentials if repeated" }
  ],

  "retry_strategy": { "retries": 3, "backoff": "exponential", "dead_letter":
"u_int_dlq" },
  "rate_limits": "Max 60 requests/min",
  "logging_monitoring": ["Integration log table u_int_log", "MID server logs (if
used)"],

  "sample_request": "{ \"summary\": \"...\" }",
  "sample_response": "{ \"id\": \"JIRA-123\" }"
}
```

JS generator

```
async function generateIntegrationSpecification(data) {
  const { Document, Packer, Table, TableRow, WidthType } = getDocx();
  const { C, para, h1, h2, bullets, hdrCell, dataCell, statusCell, codePara } =
makeDocHelpers();

  const name = safeStr(data.integration_name || "Integration Specification");
  const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
```

```
  const author = safeStr(data.author || "Author");

  const systems = data.systems || {};
  const envs = data.environments || [];
  const auth = data.authentication || {};
  const endpoints = data.endpoints || [];
  const mapping = data.field_mapping || [];
  const errors = data.error_handling || [];
  const retry = data.retry_strategy || {};

  const children = [
    h1(`${name} - Integration Specification`),
    para(`Version: ${safeStr(data.version || "")}`, false, C.GRAY, 18),
    para(`Created: ${createdDate}`, false, C.GRAY, 18),
    para(`Author: ${author}`, false, C.GRAY, 18),
    para("")
  ];

  children.push(h2("Overview"));
  children.push(...bullets([
    `Direction: ${safeStr(data.direction)}`,
    `Source System: ${safeStr(systems.source)}`,
    `Target System: ${safeStr(systems.target)}`
  ].filter(Boolean)));

  if (envs.length) {
    children.push(para(""), h2("Environments"), ...bullets(envs));
  }

  children.push(para(""), h2("Authentication"));
  children.push(...bullets([
    `Method: ${safeStr(auth.method)}`,
    safeStr(auth.details)
  ].filter(Boolean)));

  if (endpoints.length) {
    children.push(para(""), h2("Endpoints"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [2200, 1200, 3600, 2360],
      rows: [
        new TableRow({ children: [hdrCell("Name", 2200), hdrCell("Method", 1200),
hdrCell("URL", 3600), hdrCell("Purpose", 2360)] }),
        ...endpoints.map((e, i) => new TableRow({
          children: [
            dataCell(e.name || "", 2200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(e.method || "", 1200, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(e.url || "", 3600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(e.purpose || "", 2360, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }
```

```
  if (mapping.length) {
    children.push(para(""), h2("Field Mapping"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [2800, 2800, 2000, 1760],
      rows: [
        new TableRow({ children: [hdrCell("Source Field", 2800), hdrCell("Target
Field", 2800), hdrCell("Transform", 2000), hdrCell("Notes", 1760)] }),
        ...mapping.map((m, i) => new TableRow({
          children: [
            dataCell(m.source_field || "", 2800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(m.target_field || "", 2800, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(m.transform || "", 2000, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(m.notes || "", 1760, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }

  if (errors.length) {
    children.push(para(""), h2("Error Handling"));
    children.push(new Table({
      width: { size: 9360, type: WidthType.DXA },
      columnWidths: [2600, 3600, 3160],
      rows: [
        new TableRow({ children: [hdrCell("Scenario", 2600), hdrCell("Behavior",
3600), hdrCell("Owner Action", 3160)] }),
        ...errors.map((e, i) => new TableRow({
          children: [
            dataCell(e.scenario || "", 2600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(e.behavior || "", 3600, i % 2 ? C.LIGHT_GRAY : C.WHITE),
            dataCell(e.owner_action || "", 3160, i % 2 ? C.LIGHT_GRAY : C.WHITE)
          ]
        }))
      ]
    }));
  }

  children.push(para(""), h2("Retry / Rate Limits / Monitoring"));
  children.push(...bullets([
    retry.retries != null ? `Retries: ${safeStr(retry.retries)}` : "",
    retry.backoff ? `Backoff: ${safeStr(retry.backoff)}` : "",
    retry.dead_letter ? `Dead Letter: ${safeStr(retry.dead_letter)}` : "",
    data.rate_limits ? `Rate Limits: ${safeStr(data.rate_limits)}` : ""
  ].filter(Boolean)));

  if (Array.isArray(data.logging_monitoring) && data.logging_monitoring.length) {
    children.push(para("Logging & Monitoring:", true));
    children.push(...bullets(data.logging_monitoring));
  }

  if (data.sample_request) {
```

```
    children.push(para(""), h2("Sample Request"),
codePara(safeStr(data.sample_request)));
  }
  if (data.sample_response) {
    children.push(para(""), h2("Sample Response"),
codePara(safeStr(data.sample_response)));
  }

  const doc = new Document({ sections: [{ properties: {}, children }] });
  const blob = await Packer.toBlob(doc);
  return { blob, filename: `${name}_Integration_Spec.docx`.replace(/[^a-z0-
9._-]/gi, "_") };
}
```

Template 7) ServiceNow Enhancement / User Story Technical Design

JSON

```
{
  "story_id": "STRY0010091",
  "title": "Auto-route incidents based on CI + Service",
  "created_date": "2026-04-25",
  "author": "Your Name",
  "summary": "Implement routing logic using mapping table to reduce assignment
delays.",
  "in_scope": ["Flow/BR routing", "Mapping table", "Unit + UAT support"],
  "out_of_scope": ["UI redesign"],
  "assumptions": ["CMDB is accurate for phase-1 services"],
  "acceptance_criteria": ["Routes within 30 seconds", "Fallback applies if CI
missing"],
  "design": { "overview": "Flow Designer routes; fallback business rule for edge
cases." },
  "data_model_changes": [
    { "table": "u_ci_service_map", "change": "New table", "details": "CI+Service -
> Assignment Group" }
  ],
  "implementation_components": [
    { "type": "Flow", "name": "INC Auto Route", "description": "Routes on
create/update" }
  ],
  "security": [{ "item": "ACL", "details": "Only ITIL can edit mapping table" }],
  "integrations": [{ "system": "None", "details": "No external integration" }],
  "testing": { "test_cases": [{ "id": "TC01", "scenario": "CI match", "expected":
"Correct group set" }] },
  "deployment_notes": ["Update set naming: STRY0010091_Routing"],
  "risks": [{ "description": "Bad CMDB may misroute", "impact": "High",
"mitigation": "Limit scope initially" }],
```

```
  "open_questions": ["Which services are in phase 1?"]
}
```